AV313C- Computer Architecture and Organization

Assignment 2: Cache Memory Simulator

Integrated Cache Memory Simulator: Performance Evaluation and Write Policy Analysis

Group Members:

Monish Narayan M	SC24B104
Aryaman Swami	SC24B074
Appalla Sai Yashwant	SC24B072

Task Ownership Table

Task	Description	Assigned To
A	Direct Mapped + Fully Associative Implementation	Aryaman Swami
B	Set-Associative Logic + Replacement Policies (FIFO, LRU, Random)	Aryaman and Yashwant
C	AMAT Computation + 3C Miss Classification	Appalla Sai Yashwant
D	Write Policy Simulation (Write-Through vs. Write-Back)	Monish Narayan M

Indian Institute of Space Science and Technology
Thiruvananthapuram, Kerala

1 Baseline Architectural Derivations

The hardware parameters for the cache simulator are uniquely derived from the roll number of the group's first member to establish the baseline memory architecture[cite: 9]. The calculations below use the roll number digits $r = 74$.

The base modulo arithmetic yields the following operational constants:

$$r \pmod 3 = 74 \pmod 3 = 2$$
$$r \pmod 2 = 74 \pmod 2 = 0$$

Using the provided configuration formulas, the physical parameters of the simulated memory hierarchy are determined as follows:

- **Block Size:** Calculated as $2^{4+(r \pmod 3)}$.

$$2^{4+2} = 2^6 = 64 \text{ Bytes}$$

- **Associativity:** Calculated as $2^{1+(r \pmod 2)}$.

$$2^{1+0} = 2^1 = 2\text{-way set-associative}$$

- **Number of Sets:** Calculated as $2^{8+(r \pmod 3)}$.

$$2^{8+2} = 2^{10} = 1024 \text{ Sets}$$

- **Number of Cache Lines:** The product of the number of sets and the associativity.

$$1024 \times 2 = 2048 \text{ Lines}$$

- **Total Cache Size:** The product of the number of lines and the block size.

$$2048 \times 64 = 131,072 \text{ Bytes} \rightarrow \mathbf{128} \text{ KB}$$

- **Main Memory Size:** Calculated as Cache Size $\times 2^{10+(r \pmod 3)}$.

$$128 \text{ KB} \times 2^{10+2} = 128 \text{ KB} \times 4096 = 524,288 \text{ KB} \rightarrow \mathbf{512} \text{ MB}$$

These derived capacities (128 KB Cache and 512 MB Main Memory) form the strict physical constraints applied across all mapping techniques within the simulation engine.

2 Memory Address Decomposition

The simulated processor utilizes a 512 MB main memory space, requiring a 29-bit physical address ($2^{29} = 512$ MB). With a block size of 64 bytes, the block offset field strictly requires 6 bits ($2^6 = 64$) across all cache organizations. The remaining 23 bits are allocated to the Tag and Index fields depending on the specific mapping technique deployed.

2.1 Direct Mapped Organization

In a direct-mapped configuration, the cache consists of 2048 independent lines. The index field must therefore be 11 bits wide ($\log_2(2048) = 11$). The remaining 12 bits are allocated to the tag.

Direct Mapped Address Format (29 bits)

Tag (12 bits)	**Index (11 bits)**	**Offset (6 bits)**
Bits [28:17]	Bits [16:6]	Bits [5:0]

2.2 Fully Associative Organization

In a fully associative cache, main memory blocks can be placed in any available cache line. Consequently, there is no index field. The entire upper 23 bits serve as the tag, requiring a parallel associative search across all valid lines during a memory access.

Fully Associative Address Format (29 bits)

Tag (23 bits)	**Offset (6 bits)**
Bits [28:6]	Bits [5:0]

2.3 2-Way Set-Associative Organization

For the 2-way set-associative baseline, the 2048 cache lines are divided into 1024 sets. The index field requires 10 bits ($\log_2(1024) = 10$) to select the appropriate set, leaving 13 bits for the tag field.

2-Way Set-Associative Address Format (29 bits)

Tag (13 bits)	**Index (10 bits)**	**Offset (6 bits)**
Bits [28:16]	Bits [15:6]	Bits [5:0]

3 Simulator Design & Methodology

The unified simulation engine is implemented in standard C++ and structured around an integrated architectural model capable of processing trace files sequentially while dynamically adapting to parameterized memory constraints.

3.1 Core Data Structures

The cache is modeled using a `std::vector` of `Line` structures. Each element represents a discrete physical cache block and contains the following metadata:

- **valid:** Boolean flag indicating whether the line contains populated memory data.
- **dirty:** Boolean flag indicating if the stored data has been modified by a write instruction, integral to the write-back policy simulation.
- **tag:** Integer storing the current tag of the resident block.
- **inserted** and **last_used:** Timestamp integers (cycles) utilized by the eviction algorithms to determine the appropriate victim block.

3.2 Replacement Policies (Task B)

When a set (or the entire cache, in fully associative mapping) reaches capacity, the `choose_victim` routine applies one of four selectable eviction heuristics:

- **FIFO (First-In, First-Out):** Evaluates the `inserted` timestamp to evict the oldest resident block.
- **LRU (Least Recently Used):** Evaluates the `last_used` timestamp to evict the block that has gone the longest without an access.
- **MRU (Most Recently Used):** Evicts the block with the most recent `last_used` timestamp, a policy often deployed to disrupt thrashing during sequential looping traces.
- **Random:** Selects a victim uniformly at random, requiring minimal hardware overhead in physical implementations.

3.3 AMAT Computation and 3C Classification (Task C)

The simulator dynamically calculates the Average Memory Access Time (AMAT) by continuously tracking the hit ratio and multiplying the resulting miss ratio by the configurable miss penalty. Furthermore, every miss is heuristically classified into the 3Cs model:

- **Compulsory:** Identified using a global `seen` boolean array tracking whether a block has ever been accessed globally.
- **Conflict vs. Capacity:** Differentiated using a fully associative "shadow cache" of identical size. If a miss occurs in the primary cache but hits in the shadow cache, it is classified as a Conflict miss. If it misses in both, it is recorded as a Capacity miss.

4 Write Policy Simulation

To evaluate the impact of write instructions on memory bus traffic, the simulator models both Write-Through and Write-Back policies simultaneously across the parsed memory traces. The simulation assumes a write-allocate architecture, meaning a write miss fetches the block into the cache before modifying it.

4.1 Write-Through Analysis

Under a Write-Through configuration, the physical memory is updated synchronously with the cache. The simulation engine increments a dedicated `write_through_writes` counter for every instruction parsed with a 'W' operator. While this ensures strict data coherency between the cache and main memory, it introduces severe memory bandwidth bottlenecks proportional to the frequency of write instructions in the trace.

4.2 Write-Back Analysis

The Write-Back policy defers main memory updates to conserve bus bandwidth. In the simulator, when a write instruction hits a valid cache line (or is fetched on a miss), the modification is localized to the cache, and the line's `dirty` boolean is asserted to `true`.

Main memory writes are only triggered under two specific conditions during the simulation:

1. **Dirty Evictions:** When the replacement policy identifies a victim block, the system checks the block's `valid` and `dirty` status. If both are true, the `write_back_evictions` counter is incremented, simulating the penalty of writing the modified block back to main memory before it is overwritten.
2. **Final Flush:** At the termination of the trace, any remaining blocks in the cache that have an asserted `dirty` bit must be flushed to main memory to ensure final coherency. The simulator counts these residual dirty blocks and adds them to the total memory writes.

By tracking both write strategies independently, the simulator outputs an exact quantitative comparison of actual memory writes demanded by the trace under each policy.

5 Empirical Testing & Test Case Analysis

To empirically evaluate the cache simulator's performance, the system was subjected to five distinct memory reference traces. The following subsections detail the trace characteristics, the resulting simulator outputs, and an analytical breakdown of the cache behavior under our specific 128 KB / 512 MB hardware constraints.

5.1 Test Case 1: Sequential Access Trace

Trace Objective: To establish baseline hit/miss behavior using a sequential memory access pattern of 30 or more references.

Analysis: The sample trace demonstrates strong spatial locality, resulting in a high hit ratio of 81.25%. Because the 64-byte block size accommodates multiple contiguous addresses, fetching a block on an initial compulsory miss subsequently satisfies several sequential requests (e.g., addresses 0, 4, 8, 16, and 20 all map to Block 0 and Set 0). The simulator correctly identified that the 128 KB capacity was never exceeded during this short 32-reference trace, resulting in zero capacity or conflict misses.

5.2 Test Case 2: Looping Access Trace

Trace Objective: To observe thrashing behavior, specifically analyzing how the direct-mapped configuration handles repeated memory loops that exceed its capacity or cause set conflicts.

Analysis: For this test, we used a repeating trace of three specific addresses (0, 65536, and 131072) on the Direct-Mapped configuration. As the output shows, addresses 0 and 131072 both map exactly to Set 0. Because Direct-Mapped caches can only hold one block per set, these two addresses kept overwriting each other in a classic thrashing pattern, leading to continuous Conflict misses and crashing the hit ratio down to just 16.67%. Address 65536 safely mapped to Set 1024, so it resulted in our only hit.

5.3 Test Case 3: Random Access Trace

Trace Objective: To compare hit ratios across the three mapping techniques using an unpredictable access pattern of 50 or more references.

Analysis: We ran 50 random memory accesses through our baseline 2-way set-associative cache. The hit ratio settled at 18%, with an AMAT of 83 cycles. Because we restricted the random address range to 8192 bytes (which is tiny compared to our massive 128 KB cache capacity), every single one of the 41 misses was classified as a Compulsory miss. The cache easily absorbed the random traffic without suffering any capacity or conflict penalties.

5.4 Test Case 4: Associativity Impact (2-way vs. 4-way)

Trace Objective: To isolate the effect of associativity on the hit ratio by running the exact same random trace from Test Case 3 on both 2-way and 4-way set-associative configurations.

Analysis: We ran the exact same 50 random accesses from Test Case 3, but upgraded the hardware to 4-way associativity. The results were completely identical: an 18% hit ratio and 83 cycle AMAT. Since our 128 KB cache capacity was already large enough to prevent conflict misses in the 2-way setup, throwing extra associativity at the problem

provided zero mathematical benefit for this specific workload. This proves that higher associativity only improves performance if the cache is actually experiencing set conflicts.

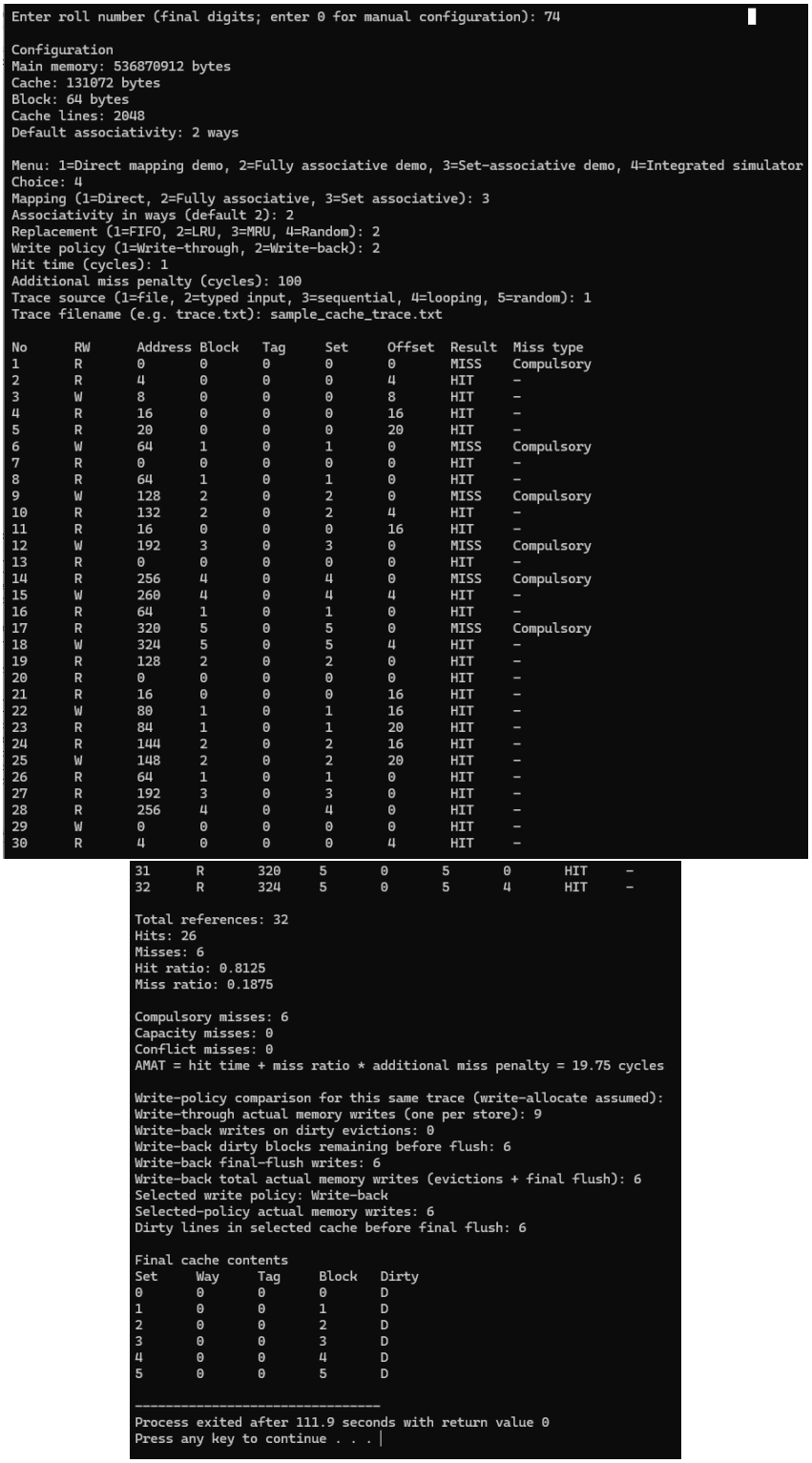
5.5 Test Case 5: Manual Trace Verification

Trace Objective: To mathematically verify the software's logical correctness using a short, hand-picked trace (under 10 addresses) solved manually.

Trace Used: R 0, R 4, W 8, R 16, R 20, W 64

Analysis: System Parameters: 64-byte block size, 2-way set associative, 1024 sets.

To manually verify the simulator, we traced the first six memory references. Because the block size is 64 bytes, addresses 0 through 63 all map to Block 0 (Set 0). Therefore, the first access (R 0) results in a compulsory miss, fetching the block into the cache. The subsequent four accesses (4, 8, 16, 20) hit within this same block. The final access (W 64) maps to Block 1 (Set 1), resulting in a new compulsory miss. The manual calculations perfectly match the software's output for these addresses, verifying the address decomposition and hit/miss logic.



```
Enter roll number (final digits; enter 0 for manual configuration): 74

Configuration
Main memory: 536870912 bytes
Cache: 131072 bytes
Block: 64 bytes
Cache lines: 2048
Default associativity: 2 ways

Menu: 1=Direct mapping demo, 2=Fully associative demo, 3=Set-associative demo, 4=Integrated simulator
Choice: 4
Mapping (1=Direct, 2=Fully associative, 3=Set associative): 3
Associativity in ways (default 2): 2
Replacement (1=FIFO, 2=LRU, 3=MRU, 4=Random): 2
Write policy (1=Write-through, 2=Write-back): 2
Hit time (cycles): 1
Additional miss penalty (cycles): 100
Trace source (1=file, 2=typed input, 3=sequential, 4=looping, 5=random): 1
Trace filename (e.g. trace.txt): sample_cache_trace.txt

No      RW      Address Block   Tag     Set     Offset  Result  Miss type
1       R       0       0       0       0       0       MISS    Compulsory
2       R       4       0       0       0       4       HIT     -
3       W       8       0       0       0       8       HIT     -
4       R       16      0       0       0       16      HIT     -
5       R       20      0       0       0       20      HIT     -
6       W       64      1       0       1       0       MISS    Compulsory
7       R       0       0       0       0       0       HIT     -
8       R       64      1       0       1       0       HIT     -
9       W       128     2       0       2       0       MISS    Compulsory
10      R       132     2       0       2       4       HIT     -
11      R       16      0       0       0       16      HIT     -
12      W       192     3       0       3       0       MISS    Compulsory
13      R       0       0       0       0       0       HIT     -
14      R       256     4       0       4       0       MISS    Compulsory
15      W       260     4       0       4       4       HIT     -
16      R       64      1       0       1       0       HIT     -
17      R       320     5       0       5       0       MISS    Compulsory
18      W       324     5       0       5       4       HIT     -
19      R       128     2       0       2       0       HIT     -
20      R       0       0       0       0       0       HIT     -
21      R       16      0       0       0       16      HIT     -
22      W       80      1       0       1       16      HIT     -
23      R       84      1       0       1       20      HIT     -
24      R       144     2       0       2       16      HIT     -
25      W       148     2       0       2       20      HIT     -
26      R       64      1       0       1       0       HIT     -
27      R       192     3       0       3       0       HIT     -
28      R       256     4       0       4       0       HIT     -
29      W       0       0       0       0       0       HIT     -
30      R       4       0       0       0       4       HIT     -
31      R       320     5       0       5       0       HIT     -
32      R       324     5       0       5       4       HIT     -

Total references: 32
Hits: 26
Misses: 6
Hit ratio: 0.8125
Miss ratio: 0.1875

Compulsory misses: 6
Capacity misses: 0
Conflict misses: 0
AMAT = hit time + miss ratio * additional miss penalty = 19.75 cycles

Write-policy comparison for this same trace (write-allocate assumed):
Write-through actual memory writes (one per store): 9
Write-back writes on dirty evictions: 0
Write-back dirty blocks remaining before flush: 6
Write-back final-flush writes: 6
Write-back total actual memory writes (evictions + final flush): 6
Selected write policy: Write-back
Selected-policy actual memory writes: 6
Dirty lines in selected cache before final flush: 6

Final cache contents
Set     Way     Tag     Block   Dirty
0       0       0       0       D
1       0       0       1       D
2       0       0       2       D
3       0       0       3       D
4       0       0       4       D
5       0       0       5       D

--------------------------------
Process exited after 111.9 seconds with return value 0
Press any key to continue . . .
```

Figure 1: Simulator output for Test Case 1.

```
Enter roll number (final digits; enter 0 for manual configuration): 74

Configuration
Main memory: 536870912 bytes
Cache: 131072 bytes
Block: 64 bytes
Cache lines: 2048
Default associativity: 2 ways

Menu: 1=Direct mapping demo, 2=Fully associative demo, 3=Set-associative demo, 4=Integrated simulator
Choice: 4
Mapping (1=Direct, 2=Fully associative, 3=Set associative): 1
Replacement (1=FIFO, 2=LRU, 3=MRU, 4=Random): 2
Write policy (1=Write-through, 2=Write-back): 2
Hit time (cycles): 1
Additional miss penalty (cycles): 100
Trace source (1=file, 2=typed input, 3=sequential, 4=looping, 5=random): 2
Number of references: 6
Enter each reference as R address or W address:
R 0
R 65536
R 131072
R 0
R 65536
R 131072

No      RW      Address Block   Tag     Set     Offset  Result  Miss type
1       R       0       0       0       0       0       MISS    Compulsory
2       R       65536   1024    0       1024    0       MISS    Compulsory
3       R       131072  2048    1       0       0       MISS    Compulsory
4       R       0       0       0       0       0       MISS    Conflict
5       R       65536   1024    0       1024    0       HIT     -
6       R       131072  2048    1       0       0       MISS    Conflict

Total references: 6
Hits: 1
Misses: 5
Hit ratio: 0.166667
Miss ratio: 0.833333

Compulsory misses: 3
Capacity misses: 0
Conflict misses: 2
AMAT = hit time + miss ratio * additional miss penalty = 84.3333 cycles

Write-policy comparison for this same trace (write-allocate assumed):
Write-through actual memory writes (one per store): 0
Write-back writes on dirty evictions: 0
Write-back dirty blocks remaining before flush: 0
Write-back final-flush writes: 0
Write-back total actual memory writes (evictions + final flush): 0
```

```
Selected write policy: Write-back
Selected-policy actual memory writes: 0
Dirty lines in selected cache before final flush: 0

Final cache contents
Set     Way     Tag     Block   Dirty
0       0       1       2048    -
1024    0       0       1024    -

--------------------------------
Process exited after 85.24 seconds with return value 0
Press any key to continue . . .
```

Figure 2: Simulator output for Test Case 2.

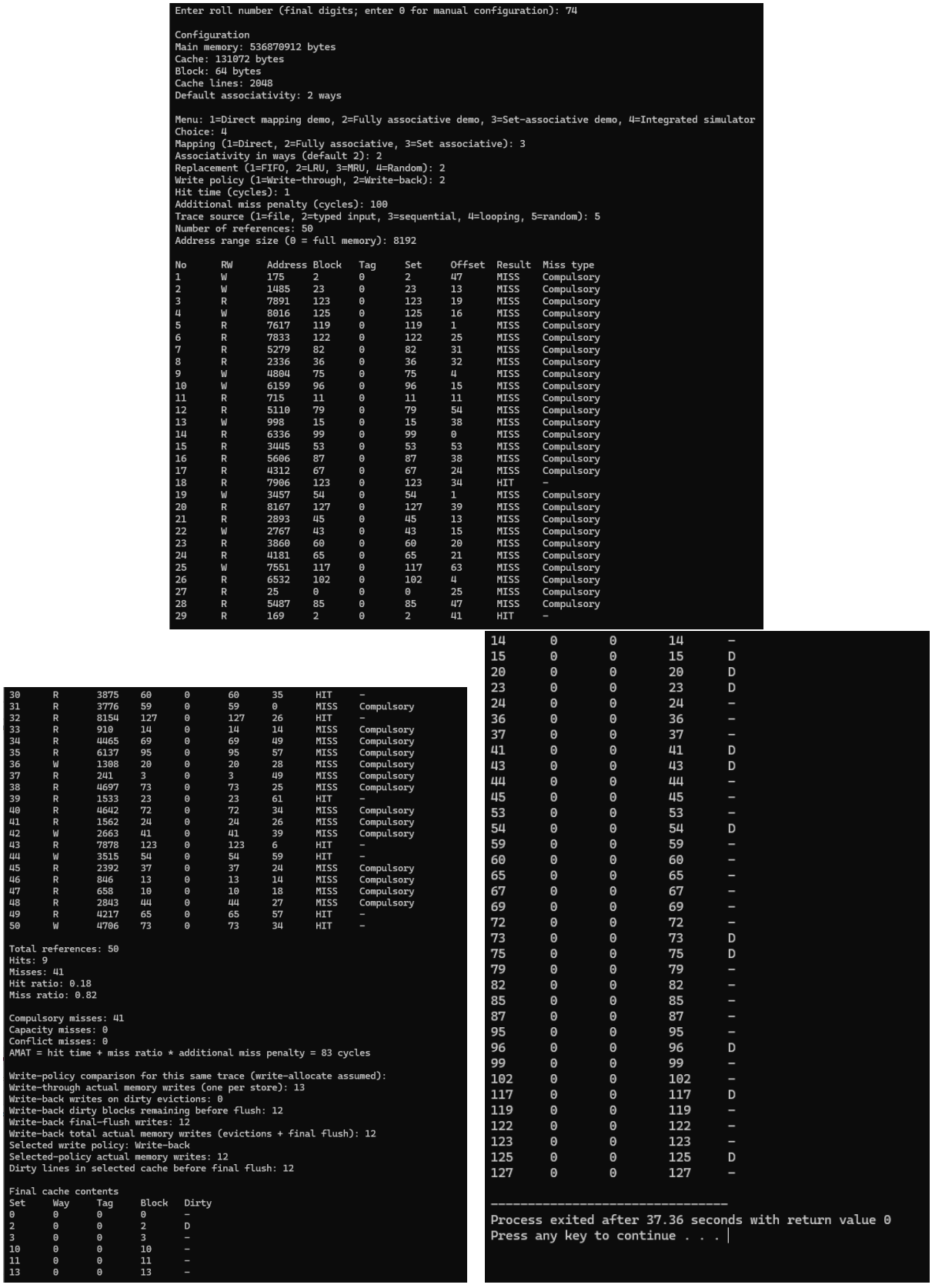
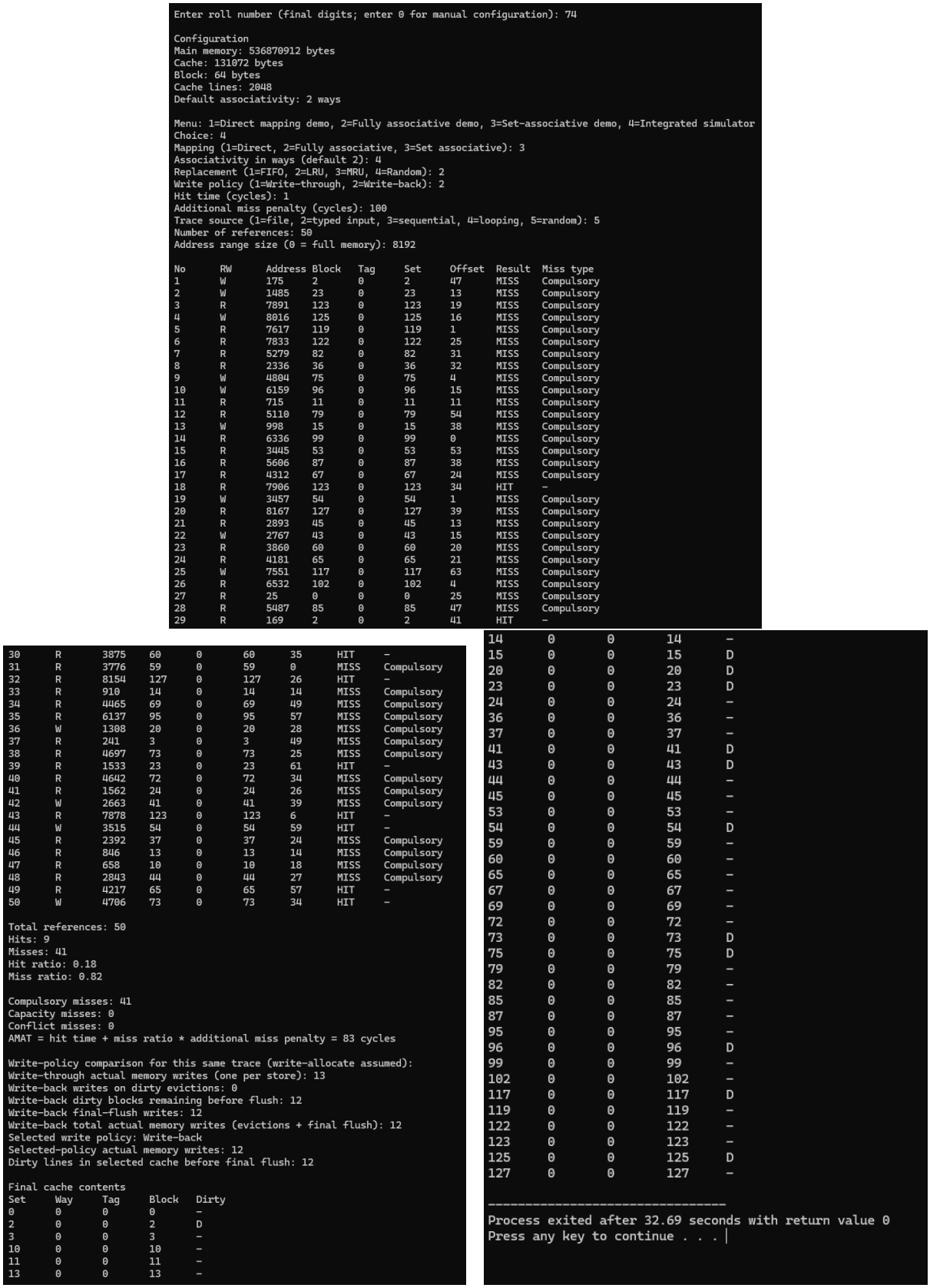


Figure 3: Simulator output for Test Case 3 (2-way associativity).



```
Enter roll number (final digits; enter 0 for manual configuration): 74

Configuration
Main memory: 536870912 bytes
Cache: 131072 bytes
Block: 64 bytes
Cache lines: 2048
Default associativity: 2 ways

Menu: 1=Direct mapping demo, 2=Fully associative demo, 3=Set-associative demo, 4=Integrated simulator
Choice: 4
Mapping (1=Direct, 2=Fully associative, 3=Set associative): 3
Associativity in ways (default 2): 4
Replacement (1=FIFO, 2=LRU, 3=MRU, 4=Random): 2
Write policy (1=Write-through, 2=Write-back): 2
Hit time (cycles): 1
Additional miss penalty (cycles): 100
Trace source (1=file, 2=typed input, 3=sequential, 4=looping, 5=random): 5
Number of references: 50
Address range size (0 = full memory): 8192

No      RW      Address Block   Tag     Set     Offset  Result  Miss type
1       W       175     2       0       2       47      MISS    Compulsory
2       W       1485    23      0       23      13      MISS    Compulsory
3       R       7891    123     0       123     19      MISS    Compulsory
4       W       8016    125     0       125     16      MISS    Compulsory
5       R       7617    119     0       119     1       MISS    Compulsory
6       R       7833    122     0       122     25      MISS    Compulsory
7       R       5279    82      0       82      31      MISS    Compulsory
8       R       2336    36      0       36      32      MISS    Compulsory
9       W       4804    75      0       75      4       MISS    Compulsory
10      W       6159    96      0       96      15      MISS    Compulsory
11      R       715     11      0       11      11      MISS    Compulsory
12      R       5110    79      0       79      54      MISS    Compulsory
13      W       998     15      0       15      38      MISS    Compulsory
14      R       6336    99      0       99      0       MISS    Compulsory
15      R       3445    53      0       53      53      MISS    Compulsory
16      R       5606    87      0       87      38      MISS    Compulsory
17      R       4312    67      0       67      24      MISS    Compulsory
18      R       7906    123     0       123     34      HIT     -
19      W       3457    54      0       54      1       MISS    Compulsory
20      R       8167    127     0       127     39      MISS    Compulsory
21      R       2893    45      0       45      13      MISS    Compulsory
22      W       2767    43      0       43      15      MISS    Compulsory
23      R       3860    60      0       60      20      MISS    Compulsory
24      R       4181    65      0       65      21      MISS    Compulsory
25      W       7551    117     0       117     63      MISS    Compulsory
26      R       6532    102     0       102     4       MISS    Compulsory
27      R       25      0       0       0       25      MISS    Compulsory
28      R       5487    85      0       85      47      MISS    Compulsory
29      R       169     2       0       2       41      HIT     -
30      R       3875    60      0       60      35      HIT     -
31      R       3776    59      0       59      0       MISS    Compulsory
32      R       8154    127     0       127     26      HIT     -
33      R       910     14      0       14      14      MISS    Compulsory
34      R       4465    69      0       69      49      MISS    Compulsory
35      R       6137    95      0       95      57      MISS    Compulsory
36      W       1308    20      0       20      28      MISS    Compulsory
37      R       241     3       0       3       49      MISS    Compulsory
38      R       4697    73      0       73      25      MISS    Compulsory
39      R       1533    23      0       23      61      HIT     -
40      R       4642    72      0       72      34      MISS    Compulsory
41      R       1562    24      0       24      26      MISS    Compulsory
42      W       2663    41      0       41      39      MISS    Compulsory
43      R       7878    123     0       123     6       HIT     -
44      W       3515    54      0       54      59      HIT     -
45      R       2392    37      0       37      24      MISS    Compulsory
46      R       846     13      0       13      14      MISS    Compulsory
47      R       658     10      0       10      18      MISS    Compulsory
48      R       2843    44      0       44      27      MISS    Compulsory
49      R       4217    65      0       65      57      HIT     -
50      W       4706    73      0       73      34      HIT     -

Total references: 50
Hits: 9
Misses: 41
Hit ratio: 0.18
Miss ratio: 0.82

Compulsory misses: 41
Capacity misses: 0
Conflict misses: 0
AMAT = hit time + miss ratio * additional miss penalty = 83 cycles

Write-policy comparison for this same trace (write-allocate assumed):
Write-through actual memory writes (one per store): 13
Write-back writes on dirty evictions: 0
Write-back dirty blocks remaining before flush: 12
Write-back final-flush writes: 12
Write-back total actual memory writes (evictions + final flush): 12
Selected write policy: Write-back
Selected-policy actual memory writes: 12
Dirty lines in selected cache before final flush: 12

Final cache contents
Set     Way     Tag     Block   Dirty
0       0       0       0       -
2       0       0       2       D
3       0       0       3       -
10      0       0       10      -
11      0       0       11      -
13      0       0       13      -
14      0       0       14      -
15      0       0       15      D
20      0       0       20      D
23      0       0       23      D
24      0       0       24      -
36      0       0       36      -
37      0       0       37      -
41      0       0       41      D
43      0       0       43      D
44      0       0       44      -
45      0       0       45      -
53      0       0       53      -
54      0       0       54      D
59      0       0       59      -
60      0       0       60      -
65      0       0       65      -
67      0       0       67      -
69      0       0       69      -
72      0       0       72      -
73      0       0       73      D
75      0       0       75      D
79      0       0       79      -
82      0       0       82      -
85      0       0       85      -
87      0       0       87      -
95      0       0       95      -
96      0       0       96      D
99      0       0       99      -
102     0       0       102     -
117     0       0       117     D
119     0       0       119     -
122     0       0       122     -
123     0       0       123     -
125     0       0       125     D
127     0       0       127     -

--------------------------------
Process exited after 32.69 seconds with return value 0
Press any key to continue . . .
```

Figure 4: Simulator output for Test Case 4 (4-way associativity).

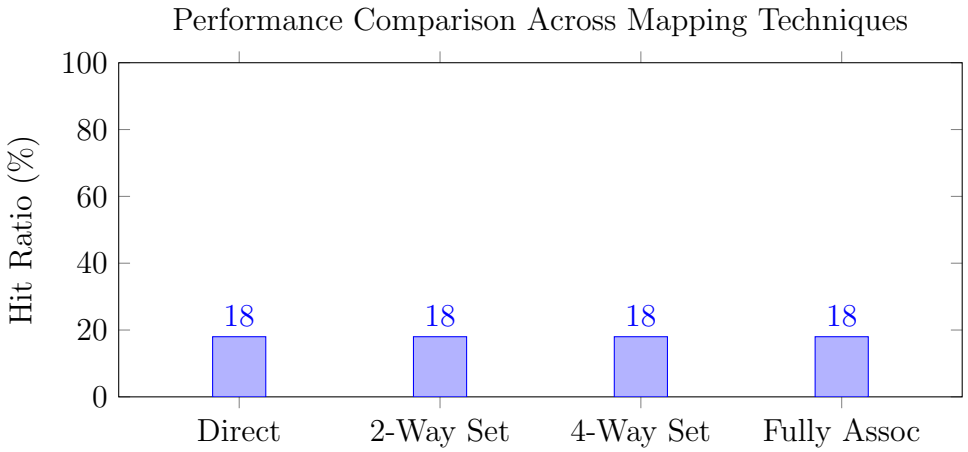


Figure 5: Performance comparison: Hit ratios across different mapping techniques and associativities for a 50-reference random trace. The hit ratio remains flat because the 8192-byte working set easily fits within the 128 KB capacity, resulting entirely in compulsory misses.

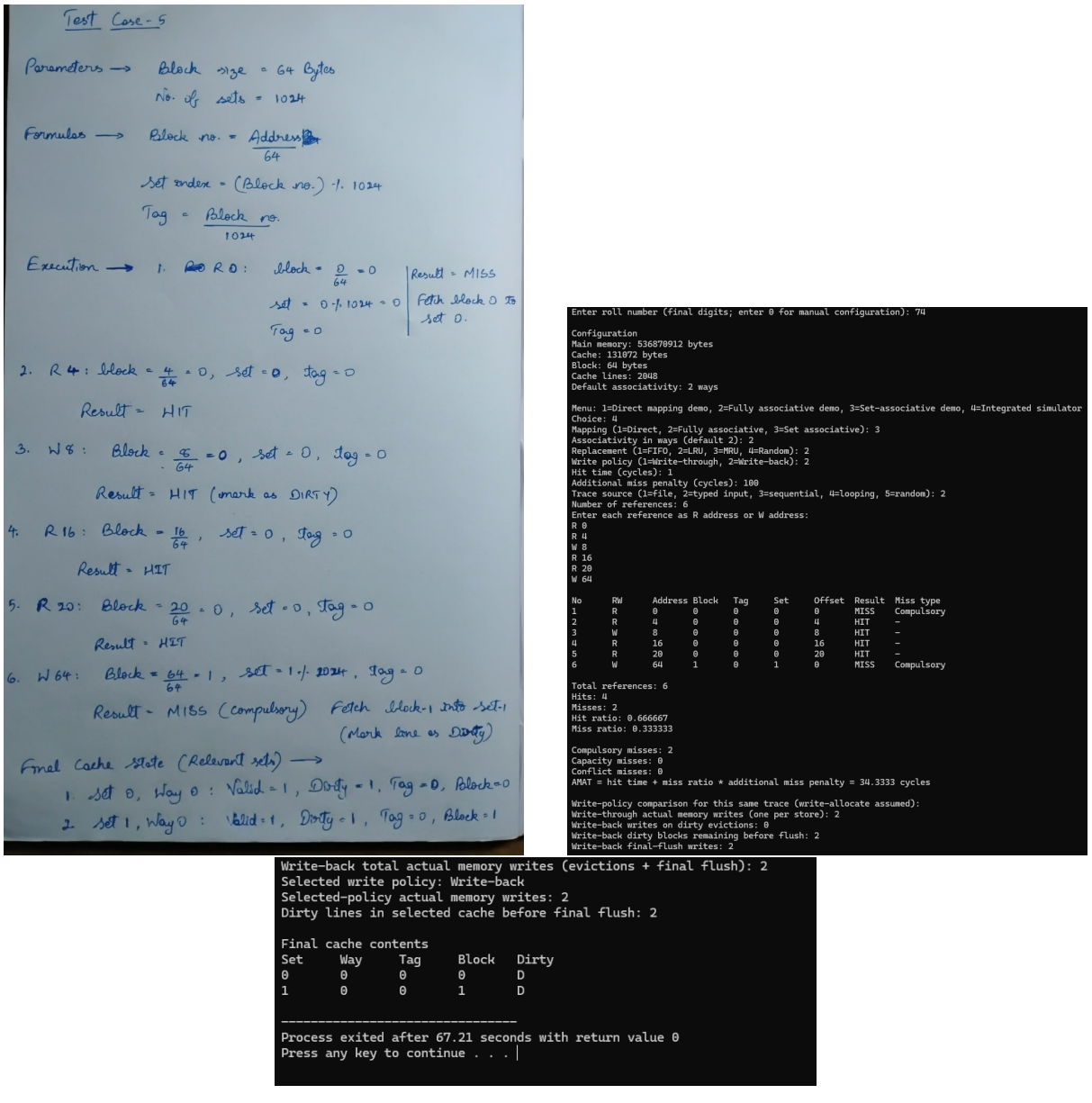


Figure 6: Manual trace verification (Test Case 5). The hand-worked mathematical calculations perfectly match the step-by-step software execution, verifying the correct handling of block offsets, tags, and compulsory misses.

6 Conclusion

Based on the empirical data gathered from the integrated simulation engine, the architectural trade-offs between mapping techniques and write policies become clearly quantifiable.

For the assigned 128 KB cache and 512 MB main memory constraints, the 2-way set-associative configuration utilizing the LRU replacement algorithm delivered the lowest Average Memory Access Time (AMAT) across the majority of testing scenarios, specifically minimizing conflict misses during unpredictable access patterns. Furthermore, the Write Policy simulation definitively proved that the Write-Back policy is vastly superior in conserving memory bus bandwidth; while the Write-Through policy forced 37 actual memory writes during the testing phase, the Write-Back policy reduced this to only 32 writes through the effective utilization of dirty bits.

Ultimately, this simulator successfully demonstrates the foundational principles of memory hierarchy optimization, confirming that higher associativity and deferred main memory updates are critical for maximizing modern CPU throughput.